

LECTURE 23-TRAP SCHEDULING,
CONTEXT SWITCHING & PROCESS
CREATION

Trap Handling in xv6

- A **trap** is when the CPU switches from **user mode** to **kernel mode**
- The following events cause a user process to “trap” into the kernel (**xv6** refers to all these events as **traps**):
 - **System calls** (requests by user for OS services)
 - **Interrupts** (external device wants attention)
 - **Program fault** (illegal action by program)
- When the above events happen, the CPU triggers a **trap** into supervisor mode.
 - In xv6 on **RISC-V**, a **system call** is invoked using the `ecall` instruction (seen in `usys.S`).
 - For **hardware interrupts**, a device sends a signal to the CPU, which causes a trap.
 - The cause of the trap is identified using the `scause` register.
- Each trap has a cause code (from `scause`), indicating the type of trap:
 - E.g., **system call** has a different `scause` value than a **keyboard interrupt**.

Trap Handling in RISC-V (xv6)

- Before Trap:

- User program is running in **user mode**

- `sepc` holds the current PC (program counter)

- `sp` points to the user stack

- Then a trap occurs, possible causes:

- **System call:** User code executes `ecall`

- **Interrupt:** e.g., timer interrupt

- **Exception:** e.g., illegal memory access

Steps Performed by CPU on Trap

- **Trap is taken:**

- CPU switches to **supervisor mode**
- Saves:
 - `sepc` (faulting PC)
 - `sstatus` (status register)
 - `scause` (reason for trap)

- **Jump to Trap Handler:**

- CPU reads `stvec` → contains address of kernel trap entry (`kernelvec` in `trampoline.S`)
- Jumps to that address

- **Manual Stack Switch:**

- CPU does **not** switch stacks automatically
- xv6 sets `sp` to the **kernel stack** of the current process
- General-purpose registers saved to **trapframe**

- **Call `usertrap()`:**

- In `trap.c`, kernel handles the trap:
 - If `scause` = `syscall` → handle system call
 - If `scause` = external interrupt → call device handler
 - If `scause` = fault → kill process or handle exception

Trapframe

- A **trapframe** is a data structure that holds the complete **CPU context** of a user process at the moment a **trap** occurs (like a system call, exception, or interrupt).
- It allows the kernel to:
 - **Safely handle the trap**
 - Then **restore** the user process's exact state to resume execution
- Each process in xv6 has a trapframe located in **kernel space**, pointed to by: `struct trapframe *trapframe; // in struct proc`
- This trapframe is allocated per process during process setup and lives in the **kernel page table**, mapped specially via trampoline.

What Does the Trapframe Contain

```
struct trapframe {  
    uint64 ra;  
    uint64 sp;  
    uint64 gp;  
    uint64 tp;  
    uint64 t0, t1, t2;  
    uint64 s0, s1;  
    uint64 a0, a1, a2, a3, a4, a5, a6, a7;  
    uint64 s2, s3, s4, s5, s6, s7, s8, s9, s10, s11;  
    uint64 t3, t4, t5, t6;  
  
    // special registers  
    uint64 epc;    // user program counter at time of trap  
    uint64 kernel_sp; // kernel stack pointer  
    uint64 kernel_satp; // kernel page table  
    uint64 kernel_trap; // usertrap() function address  
    uint64 kernel_hartid; // core id  
};
```

- All general-purpose registers (a0–a7, t0–t6, s0–s11, etc.)
- Special values:
 - epc → where to resume in user code
 - kernel_sp → kernel stack
 - kernel_satp → kernel page table
 - kernel_trap → address of usertrap()

This structure ensures that **after the kernel finishes**, the user program can be **restored exactly** as it was before the trap.

Trap Entry: `uservec` in `trampoline.S`

- When the CPU traps from user mode, it jumps to **`uservec`**, an assembly routine in `trampoline.S`.
- What `uservec` Does:
 - Saves **all general-purpose registers** to the process's **trapframe**
 - Saves the user program counter from `sepc` → `trapframe->epc`
 - Switches from the **user page table** to the **kernel page table**
 - Sets the stack pointer to the kernel stack
 - Calls the kernel C function **`usertrap()`**

usertrap()

- What `usertrap()` Does:
- Confirms trap came from user mode
- Reads `scause` to determine trap type
 - If **`ecall`** → syscall handler
 - If **`interrupt`** → call device handler
 - If **`fault`** → kill process or handle
- May call `yield()` to give up CPU
- Prepares to return by calling **`usertrapret()`**

Returning to User: `usertrapret()` → `userret` → `sret`

- `usertrapret()` sets up the `trapframe` fields and switches back to user mode using the `trampoline.S` function `userret`.
- In `userret`:
 - Restores all user registers from `trapframe`
 - Sets `sepc` to resume execution
 - Sets `sstatus` to return to user mode
 - Executes **`sret`**, which switches back to user mode and resumes execution

Traps from Kernel Mode: `kernelvec`

- If a trap occurs while in **kernel mode** (e.g., a device interrupt during a syscall), the trap entry is different.
 - In this case, `stvec` is set to point to **`kernelvec`**, defined in `kernelvec.S`
 - This handler saves fewer registers and directly calls kernel-level interrupt handlers (e.g., timer, UART)

Trap via system call

- Trigger:

- Executed by user code with the **ecall** instruction
- Traps to kernel with `scause = 8` (Environment call from U-mode)

- Syscall Function: `syscall()` in `syscall.c`

- **Syscall number:** passed in `a7`
- **Arguments:** passed in `a0–a6`
- **Return value:** written to `a0`

- Example: `a0 = fork();`

Trap via device interrupts

- Trigger:
- Hardware devices (UART, Disk) send interrupts via **PLIC**
- **scause** indicates **external interrupt**
- IRQ Mapping:
 - Devices mapped to specific IRQ numbers:
 - UART0 → 10
 - VIRTIO (disk) → 1

```
int irq = plic_claim();
if (irq == UART0_IRQ)
    uartintr();          // Handle UART
else if (irq == VIRTIO0_IRQ)
    virtio_disk_intr();  // Handle
disk
plic_complete(irq);     //
Acknowledge
```

Trap via timer interrupt

- Trigger:

- **Supervisor timer interrupt** from **CLINT**
- Happens periodically for preemption
- `scause = 0x8000000000000001`

- Purpose:

- Enables **preemptive scheduling**
- Wakes up sleeping processes
- Forces running process to yield

Summary of xv6 trap handling

- **System calls** cause the CPU to execute the **ecall** instruction, while **program faults** and **hardware interrupts** cause the CPU to **take a trap** directly, transferring control to the kernel.
- The trap causes the CPU to **switch from user mode to supervisor mode**, and jump to the trap handler address stored in **stvec** (e.g., `uservec`).
- The **pre-trap CPU state** (registers, `sepc`, etc.) is saved into the **trapframe** on the kernel stack by the `uservec` code in `trampoline.S`.
- The **kernel trap handler** (`usertrap()` in C) handles the trap and later returns to user space using `sret`, restoring the user state from the `trapframe`.

Context Switching

Context switching in xv6

- Every CPU has a scheduler thread (special process that runs scheduler code)
- Scheduler goes over list of processes and switches to one of the runnable ones
- After running for some time, the process switches back to the scheduler thread, when:
 - Process has terminated
 - Process needs to sleep (e.g., blocking read system call)
 - Process yields after running for long (timer interrupt)
- Scheduler thread runs its loop and picks next process to run, and the story repeats
- Context switch only happens when process is already in kernel mode.
 - Example: P1 running, timer interrupt, P1 moves to kernel mode, switches to scheduler thread, scheduler switches to P2, P2 returns to user mode

Two main functions to understand

Function	RISC-V (xv6-riscv)	Purpose
scheduler()	scheduler()	Infinite loop, picks processes
sched()	sched()	Switch from process to scheduler

scheduler()

- It is the **main scheduling loop** that runs on **every CPU**.
- Its job is to **pick a RUNNABLE process** and switch to it.
- It runs in an infinite loop, the OS never stops scheduling.
- **Loop Forever:**
Continuously look for processes to run.
- **Enable & Disable Interrupts:**
Avoid deadlocks and race conditions.
- **Scan Process Table:**
For each process in `proc [ ]`:
 - Acquire its lock.
 - If RUNNABLE:
 - Mark RUNNING, set `c->proc`
 - `swtch()` to switch to process context.
 - Release lock.
- **If No Process Found:**
Use `wfi` (Wait For Interrupt) to idle the CPU efficiently.

```
void
scheduler(void)
{
    struct proc *p;
    struct cpu *c = mycpu();

    c->proc = 0;
    for(;;){
        // Ensure interrupts are briefly enabled to avoid deadlock
        intr_on();
        intr_off();

        int found = 0;
        for(p = proc; p < &proc[NPROC]; p++) {
            acquire(&p->lock);
            if(p->state == RUNNABLE) {
                // Run this process
                p->state = RUNNING;
                c->proc = p;
                swtch(&c->context, &p->context);

                // Process yielded; reset CPU's proc
                c->proc = 0;
                found = 1;
            }
            release(&p->lock);
        }

        if(found == 0) {
            // No runnable process; wait for interrupt
            asm volatile("wfi");
        }
    }
}
```

sched()

- Transfers control from the current process back to the scheduler.
- Used when a process **yields, sleeps, or exits**.
- **Perform Safety Checks**
Ensures the process lock is held, the process is not in the RUNNING state, interrupts are disabled, and lock depth is correct.
- **Save Interrupt State:** Remembers whether interrupts were enabled before the process started blocking.
- **Context Switch to Scheduler:** Saves the current process's CPU context and restores the scheduler's context, allowing the CPU to resume the scheduler() loop.
- **Restore Interrupt State:** After the scheduler resumes the process later, the original interrupt state is restored.

```
void
sched(void)
{
    int intena;
    struct proc *p = myproc();

    // Sanity checks before context switch
    if(!holding(&p->lock))
        panic("sched: p->lock not held");
    if(mycpu()->noff != 1)
        panic("sched: incorrect lock depth");
    if(p->state == RUNNING)
        panic("sched: process still RUNNING");
    if(intr_get())
        panic("sched: interrupts enabled");

    // Save current interrupt enable state
    intena = mycpu()->intena;

    // Context switch: save process context, restore scheduler context
    swch(&p->context, &mycpu()->context);

    // Restore interrupt enable flag
    mycpu()->intena = intena;
}
```

Who calls sched()?

- Yield: Timer interrupt occurs, process has run enough, gives up CPU
- Exit: Process has called exit, sets itself as zombie, gives up CPU
- Sleep: Process has performed a blocking action, sets itself to sleep, gives up CPU

```
// Give up the CPU for one scheduling round.
void
yield(void)
{
    struct proc *p = myproc();
    acquire(&p->lock);
    p->state = RUNNABLE;
    sched();
    release(&p->lock);
}
```

```
void
sleep(void *chan, struct spinlock *lk)
{
    struct proc *p = myproc();

    // Acquire p->lock to safely change state and call sched.
    // This prevents missing wakeups (wakeup also acquires p->lock).
    acquire(&p->lock);
    release(lk);

    // Set sleep channel and state, then switch process out.
    p->chan = chan;
    p->state = SLEEPING;
    sched();

    // Clear channel and restore locks after waking.
    p->chan = 0;
    release(&p->lock);
    acquire(lk);
}
```

struct context

- In both `scheduler()` and `sched()` functions, the function **switch** switches between two **context** structures.
- The **context** structure is a set of registers saved during a context switch to preserve the CPU state of a process.
- We must save the **sepc** register (the program counter where the process stopped) so the process can resume execution at the same instruction when scheduled again.
- The **context** is stored on the kernel stack, and the process structure (`struct proc`) maintains a pointer to this context (`p->context`).

```
1 // Saved registers for kernel context switches.
2 ✓ struct context {
3     uint64 ra;
4     uint64 sp;
5
6     // callee-saved
7     uint64 s0;
8     uint64 s1;
9     uint64 s2;
10    uint64 s3;
11    uint64 s4;
12    uint64 s5;
13    uint64 s6;
14    uint64 s7;
15    uint64 s8;
16    uint64 s9;
17    uint64 s10;
18    uint64 s11;
19 };
20
```

Context structure vs. trap frame

- **Trapframe** (`p->trapframe`) contains the saved user register state on the kernel stack for a process.
- The trapframe is saved when the CPU switches from user mode to kernel mode (e.g., `sepc` in trapframe is the program counter where the trap or syscall occurred in user code).
- The **context** structure is saved when the process switches to another process (e.g., registers saved when `swtch` is called).
- Both are stored on the kernel stack, and `struct proc` has pointers to both.
- Example: Process P1 receives a timer interrupt, saves trapframe on its kernel stack, then calls `swtch` and saves the context structure on the kernel stack.

swtch function (1)

- Both the CPU (`mycpu( )->context`) and the process (`struct proc *p->context`) maintain pointers to their **context** structures.
- `swtch` takes two arguments: the address of the old context pointer to switch from, and the new context pointer to switch to.
- When called from the **scheduler**, `swtch` is invoked with the address of the **scheduler's context** (`&mycpu( )->context`) as the old context, and the **process's context** (`p->context`) as the new context.
- When called from **sched()**, `swtch` is invoked with the address of the **process's context** (`&p->context`) as the old context, and the **scheduler's context** (`mycpu( )->context`) as the new context.

scheduler() calling swtch

- scheduler() calls swtch to switch from **scheduler to process**
- The **scheduler** is the main loop that selects a process to run.
- When it finds a process p in the RUNNABLE state, it:
 - Sets c->proc = p (assigns process to CPU).
 - Changes p->state to RUNNING.
 - Calls swtch(&c->context, &p->context).
- **What happens here?**
 - The current CPU state (scheduler's context) is saved in c->context.
 - The process's saved context (p->context) is loaded.
 - CPU begins executing the process from where it left off.
- When the process yields, sleeps, or exits, it will switch back to the scheduler.

sched() calling swtch

- When a process needs to stop running (sleep, yield, exit), it calls sched().
 - Checks that process lock is held and interrupts are disabled.
 - Calls swtch(&p->context, &mycpu()->context).
- **What happens here?**
 - The current process's CPU state is saved into p->context.
 - The CPU state saved in mycpu()->context (the scheduler's context) is restored.
 - Execution resumes in the scheduler, continuing its loop.

swtch function (2)

- What is on the kernel stack when a process/thread has just invoked the swtch?
 - Caller save registers (refer to C calling convention)
 - Return address (ra)
- What does swtch do?
 - Saves remaining callee-save registers (s0-s11, sp, ra) onto the old kernel stack (only callee-saved registers need saving)
 - Saves pointer to this context in the old process's context structure pointer
 - Switches stack pointer (sp) from old kernel stack to new kernel stack
 - sp now points to saved context of the new process
 - Restores callee-save registers from the new stack
 - Returns from the function call (restoring return address and caller-saved registers)
- What will swtch find on new kernel stack? Where does it return to?
 - Whatever was pushed when the new process gave up its CPU in the past
- Result of swtch: we switched kernel stacks from old process to new process, CPU is now executing new process code, resuming where the process gave up its CPU by calling swtch in the past

swtch function (3)

- When the `swtch` function is called, the **kernel stack** of the old process already has (from the top):
 - Return address (`ra`), and arguments to `swtch` (pointers to old and new context structures).
- The address of the old context pointer is passed in `a0`.
- The address of the new context pointer is passed in `a1`.
- `swtch` stores callee-saved registers (`ra`, `sp`, `s0-s11`) onto the kernel stack of the old process.
- After saving, the top of the stack pointer (`sp`) points to the saved context of the old process.
- The old process's context pointer (`struct context *` in `struct proc`) is updated to point to this saved context.
- Stack pointer (`sp`) is switched to point to the new process's saved context (value in `a1`).
- Registers are restored from the new process's context.
- `swtch` returns, resuming execution of the new process.
- CPU is now running the new process on its kernel stack, continuing where it left off.

```
8      .globl swtch
9      swtch:
10         sd ra, 0(a0)
11         sd sp, 8(a0)
12         sd s0, 16(a0)
13         sd s1, 24(a0)
14         sd s2, 32(a0)
15         sd s3, 40(a0)
16         sd s4, 48(a0)
17         sd s5, 56(a0)
18         sd s6, 64(a0)
19         sd s7, 72(a0)
20         sd s8, 80(a0)
21         sd s9, 88(a0)
22         sd s10, 96(a0)
23         sd s11, 104(a0)
24
25         ld ra, 0(a1)
26         ld sp, 8(a1)
27         ld s0, 16(a1)
28         ld s1, 24(a1)
29         ld s2, 32(a1)
30         ld s3, 40(a1)
31         ld s4, 48(a1)
32         ld s5, 56(a1)
33         ld s6, 64(a1)
34         ld s7, 72(a1)
35         ld s8, 80(a1)
36         ld s9, 88(a1)
37         ld s10, 96(a1)
38         ld s11, 104(a1)
39
40         ret
41
42
```

Summary of context switching in xv6

- What happens during context switch from process P1 to P2?
 - P1 goes to kernel mode and gives up CPU (timer interrupt or exit or sleep)
 - P2 is another process that is ready to run (it had given up CPU after saving context on its kernel stack in the past, but is now ready to run)
 - P1 switches to CPU scheduler thread
 - Scheduler thread finds runnable process P2 and switches to it
 - P2 returns from trap to user mode
- Process of switching from one process/thread to another
 - Save all register state (CPU context) on kernel stack of old process
 - Update context structure pointer of old process to this saved context
 - Switch from old kernel stack to new kernel stack
 - Restore register state (CPU context) from new kernel stack, and resume new process

Process creation in xv6

New process creation in xv6

- **Init process** is the first user process created by xv6 after boot-up.
- This `init` process **forks the shell**, which in turn forks other processes to run user commands.
- The `init` process is the **ancestor of all processes** in Unix-like systems.
- After `init`, all other processes are created using the **fork system call**, where a parent process spawns a child.
- The function **allocproc** is called both during:
 - the creation of the `init` process, and
 - inside the `fork` system call.
- What `allocproc` does (RISC-V):
 - Allocates a new `struct proc`, assigns **PID**, and initializes process metadata.
 - Allocates a **kernel stack** for the new process.
 - Sets up a **trapframe** (`p->trapframe`) to hold the CPU state when entering/exiting kernel mode.
 - Prepares a **context** (`p->context`) so the process can be **context switched in** by the scheduler (via `swtch`).

allocproc

- Find an unused entry in the process table (ptable) and mark it as **EMBRYO**.
- After initialization is complete, process is marked as **RUNNABLE**.
- Assign a new **PID**.
- Allocate new memory for the process's **kernel stack**.
- At the bottom of the kernel stack, leave space for the **trapframe** (struct trapframe), which will hold the saved user state when the process enters the kernel.
- Allocate and set up the **context** (struct context) just above the trapframe:
 - Set the return address (ra) in the context to point to **forkret**.
- Why forkret?
 - When this new process is scheduled for the first time, it starts executing at forkret.
 - forkret eventually calls usertrapret, which uses sepc to return to **user space**.
- allocproc builds a **hand-crafted kernel stack** to make the process look like it had previously entered the kernel due to a trap and was context-switched out.
- This allows the **scheduler** to switch it in just like any other process.

```
105 // Look in the process table for an UNUSED proc.
106 // If found, initialize state required to run in the kernel,
107 // and return with p->lock held.
108 // If there are no free procs, or a memory allocation fails, return 0.
109 static struct proc*
110 allocproc(void)
111 {
112     struct proc *p;
113
114     for(p = proc; p < &proc[NPROC]; p++) {
115         acquire(&p->lock);
116         if(p->state == UNUSED) {
117             goto found;
118         } else {
119             release(&p->lock);
120         }
121     }
122     return 0;
123
124 found:
125     p->pid = allocpid();
126     p->state = USED;
127
128     // Allocate a trapframe page.
129     if((p->trapframe = (struct trapframe *)kalloc()) == 0){
130         freeproc(p);
131         release(&p->lock);
132         return 0;
133     }
134
135     // An empty user page table.
136     p->pagetable = proc_pagetable(p);
137     if(p->pagetable == 0){
138         freeproc(p);
139         release(&p->lock);
140         return 0;
141     }
142
143     // Set up new context to start executing at forkret,
144     // which returns to user space.
145     memset(&p->context, 0, sizeof(p->context));
146     p->context.ra = (uint64)forkret;
147     p->context.sp = p->kstack + PGSIZE;
148
149     return p;
150 }
```

Init process creation

- `allocproc` has created a new process and set up its kernel stack and trapframe.
- When this process is first **scheduled**, it starts executing from the function **forkret**.
- `forkret` then calls **usertrapret**, which sets up the processor to return to **user mode** using the trapframe.
- The **trapframe** is set such that the program counter (`sepc`) points to the **first instruction of `initcode.S`**, a small user program embedded in the kernel.
- `initcode.S` runs in **user mode** and immediately makes an `exec` system call to run the real **init program** (from the file system).

```
218 // Set up first user process.
219 void
220 userinit(void)
221 {
222     struct proc *p;
223
224     p = allocproc();
225     initproc = p;
226
227     p->cwd = namei("/");
228
229     p->state = RUNNABLE;
230
231     release(&p->lock);
232 }
233
```

Init process

- **Opens standard file descriptors:**
The init process opens "console" as **stdin (0)**, then duplicates it for **stdout (1)** and **stderr (2)** using `dup(0)`.
- **These descriptors are inherited** by all child processes, since **file descriptor tables are inherited on fork**.
- **Forks a child process**, and in the child, calls `exec("sh", argv)` to **replace its image with the shell program**.
- If fork or exec fails, prints an error and exits.
- The parent (init) **waits for the shell to exit** using `wait()`.
- If the shell dies (user exits shell), the parent **restarts the shell**.
- Also **reaps other orphaned processes** (those with no living parent) by repeatedly calling `wait()` in the loop.

```
// init: Initial user-level program
```

```
#include "kernel/types.h"
#include "kernel/stat.h"
#include "kernel/spinlock.h"
#include "kernel/sleeplock.h"
#include "kernel/fs.h"
#include "kernel/file.h"
#include "user/user.h"
#include "kernel/fcntl.h"
```

```
char *argv[] = { "sh", 0 };
```

```
int main(void) {
    int pid, wpid;
```

```
    if(open("console", O_RDWR) < 0){ mknod("console", CONSOLE, 0); open("console", O_RDWR);
        dup(0); dup(0); // stdout, stderr
```

```
    for(;;){
        printf("init: starting sh\n");
        pid = fork();
        if(pid < 0){ printf("init: fork failed\n"); exit(1); }
        if(pid == 0){ exec("sh", argv); printf("init: exec sh failed\n"); exit(1); }
```

```
    for(;;){
        wpid = wait((int *)0);
        if(wpid == pid) break;
        else if(wpid < 0){ printf("init: wait error\n"); exit(1); }
    }
}
```

Forking new process

- Calls `allocproc` to allocate a new process structure.
- Copies parent's user memory into the child's page table.
- Duplicates file descriptors and working directory from parent to child.
- Copies trapframe (register state) from parent to child, so child resumes execution at the same point as parent.
- Sets child's return value (`a0` register) to 0, distinguishing child from parent.
- Marks child process state as `RUNNABLE`, ready for scheduling.
- Parent returns with child's PID; child runs later when scheduled.

```
// Create new process copying parent; set up child kernel stack to return as
from fork()
int kfork(void) {
    int i, pid;
    struct proc *np;
    struct proc *p = myproc();

    if ((np = allocproc()) == 0) return -1;

    if (uvmcopy(p->pagetable, np->pagetable, p->sz) < 0) {
        freeproc(np);
        release(&np->lock);
        return -1;
    }
    np->sz = p->sz;
    *(np->trapframe) = *(p->trapframe); // copy user registers
    np->trapframe->a0 = 0; // fork returns 0 in child

    for (i = 0; i < NOFILE; i++)
        if (p->ofile[i]) np->ofile[i] = filedup(p->ofile[i]);
    np->cwd = idup(p->cwd);
    safestrcpy(np->name, p->name, sizeof(p->name));

    pid = np->pid;
    release(&np->lock);

    acquire(&wait_lock);
    np->parent = p;
    release(&wait_lock);

    acquire(&np->lock);
    np->state = RUNNABLE;
    release(&np->lock);

    return pid;
}
```

Summary of new process creation

- A new process is created by marking an entry in the process table as `RUNNABLE` after setting up its kernel stack, memory, and trapframe.
- The kernel stack of the new process is crafted to appear as if it was previously context-switched out.
- This allows the scheduler to switch to the new process using the regular `swtch` mechanism without any special handling.



Thank You